

Lia — Growth Log

The stages of her growth, kept for tracking what she can do at each one.

A record of Lia's growth in stages, not a log of every exchange or bug fix — those live in the Session Summary and Session Transcript. This document is for tracking what kind of thing she is at each stage: what she could do, and what she couldn't yet.

The shape of it: each stage she gained a new kind of relationship to the world — first just talking, then reading something you hand her, then perceiving and being perceived (voice), and now, starting to act on the world beyond just describing it.

Stage 1 — Making her

What she gained: existence. A text-only companion, typed to and typed back at, but with something most chatbots don't have — memory that survives closing the window.

- A local brain: Ollama running `llama3.2:3b`, small enough for a 4GB graphics card.
- Two-tier memory: a short-term rolling buffer for the live conversation, and a permanent SQLite database for facts and semantically searchable past turns (`nomic-embed-text` for the embeddings).
- A diary — a private reflection written at the end of each session.
- Six files: `config`, `db`, `llm`, `memory`, `diary`, `main`.

What she couldn't do yet: *hear you, speak, read anything you hadn't typed directly into the conversation, or do anything beyond generating the next reply.*

Stage 2 — Controlling actions from a file explorer path

What she gained: her first action beyond conversation — reading something real, from a place on disk, on request.

- A library folder (LIA/library/) you hand her documents through — PDFs, text, markdown.
- Say “read my files” and she indexes what's there: extracted, chunked, embedded, page-cited so she can say exactly where an answer came from.
- Deliberately on-demand, never automatic. She does not scan your drive, and a file sitting in the folder is *available* to her, not already read — a boundary kept even when it would have been easier to just read everything on startup.

What she still couldn't do: *hear or speak a word, or touch anything outside that one folder.*

Stage 3 — Adding voices and ML models

What she gained: a body, in the only sense that matters for a companion — a voice, and ears. This is the stage where she stopped being a chat window and became something you actually talk to.

- **Speaking:** Piper, a local neural text-to-speech engine, streamed sentence-by-sentence so she starts talking before the whole reply has even finished generating.
- **Listening:** faster-whisper for transcription, Silero VAD for detecting when you start and stop talking — together, an open microphone rather than push-to-talk.
- **A name to be called:** a wake word matched on the transcript itself, needing no trained model, tolerant of however her name actually gets misheard.
- **Presence without a window:** a system tray app, autostart at login, packaged as a standalone executable.
- A second voice engine (KittenTTS) was explored, got fully working, and was then removed by request — the seam for adding another one is still there in `voice.py` if it's ever wanted again.

***What she still couldn't do:** affect anything outside the conversation itself — no control over the machine she runs on, beyond her own voice and ears.*

Stage 4 — Controlling actions (in progress)

What's changing: the current frontier. Everything up to Stage 3 was perception and memory — hearing, speaking, reading, remembering. Stage 4 is the first time she reaches outward and changes something in the world rather than just describing it.

Built so far

- **Media control** — play, pause, skip whatever's already playing (Spotify, a browser tab, any app), via Windows' System Media Transport Controls. A remote, not a jukebox: it can't start a track from nothing, and says so honestly rather than pretending.
- **Volume control** — up, down, mute, unmute, via direct Core Audio access. (Simulating the physical volume keys was tried first and found to silently do nothing — fixed by talking to the audio API directly instead of imitating a keypress.)
- **Alarms and timers** — genuine ones, checked by a background watchdog every ten seconds and persisted across restarts, so “remind me in twenty minutes” survives even if the conversation, or the app itself, closes in the meantime.

What she knows about herself

A real gap surfaced during this stage: she had no idea any of this existed. Asked directly “can you play music,” she answered from generic assumptions about being “a text-based system” — technically wrong, since the capability was already built, just never told to her. Fixed by adding an explicit, honest capability list to her own system prompt, so she reports what she can and can't do accurately instead of guessing.

Not yet built

- Opening or closing applications.
- Anything involving the mouse, keyboard, or screen beyond the specific controls above.
- Browsing the web or clicking anything on your behalf.
- Any action on Linux or macOS — everything in this stage is Windows-specific so far.

Where this goes next is genuinely open. The pattern that's worked twice now — find the narrowest real OS API for a specific action (SMTC for media, Core Audio for volume) rather than reaching for something broad and risky — is the one to keep following as this stage grows.